JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.stream

Class Collectors

java.lang.Object
    java.util.stream.Collectors

```
public final class Collectors
extends Object
```

Implementations of `Collector` that implement various useful reduction operations, such as accumulating elements into collections, summarizing elements according to various criteria, etc.

The following are examples of using the predefined collectors to perform common mutable reduction tasks:

```
// Accumulate names into a List
List<String> list = people.stream().map(Person::getName).collect(Collectors.to

// Accumulate names into a TreeSet
Set<String> set = people.stream().map(Person::getName).collect(Collectors.toCo

// Convert elements to strings and concatenate them, separated by commas
String joined = things.stream()
                      .map(Object::toString)
                      .collect(Collectors.joining(", "));

// Compute sum of salaries of employee
int total = employees.stream()
                     .collect(Collectors.summingInt(Employee::getSalary)));

// Group employees by department
Map<Department, List<Employee>> byDept
    = employees.stream()
```

```
                .collect(Collectors.groupingBy(Employee::getDepartment));

    // Compute sum of salaries by department
    Map<Department, Integer> totalByDept
        = employees.stream()
                .collect(Collectors.groupingBy(Employee::getDepartment,
                                               Collectors.summingInt(Employee::

    // Partition students into passing and failing
    Map<Boolean, List<Student>> passingFailing =
        students.stream()
                .collect(Collectors.partitioningBy(s -> s.getGrade() >= PASS_THRES
```

Since:

1.8

Method Summary

Concrete Methods

Modifier and Type	Method
static <T> **Collector**<T,?,**Double**>	**averagingDouble**(**ToDoubleFun** T> mapper)
static <T> **Collector**<T,?,**Double**>	**averagingInt**(**ToIntFunction**<
static <T> **Collector**<T,?,**Double**>	**averagingLong**(**ToLongFunctio** T> mapper)

static <T,A,R,RR> **Collector**<T,A,RR>	**collectingAndThen**(**Collector**<T,A,R> downstream, **Function**<R,RR> finisher)
static <T> **Collector**<T,?,**Long**>	**counting**()
static <T,K> **Collector**<T,?,**Map**<K,**List**<T>>>	**groupingBy**(**Function**<? super T,? extends K> classifier)
static <T,K,A,D> **Collector**<T,?,**Map**<K,D>>	**groupingBy**(**Function**<? super T,? extends K> classifier, **Collector**<? super T,A,D> downstream)

static <T,K,D,A,M extends **Map**<K,D>> **Collector**<T,?,M>	**groupingBy**(**Function**<? super K> classifier, **Supplier**<M> **Collector**<? super T,A,D> do
static <T,K> **Collector**<T,?,**ConcurrentMap**<K,**List**<T>>>	**groupingByConcurrent**(**Functi** extends K> classifier)
static <T,K,A,D> **Collector**<T,?,**ConcurrentMap**<K,D>>	**groupingByConcurrent**(**Functi** extends K> classifier, **Coll** T,A,D> downstream)

static <T,K,A,D,M extends **ConcurrentMap**<K,D>> **Collector**<T,?,M>	**groupingByConcurrent**(**Functi** extends K> classifier, **Supp** **Collector**<? super T,A,D> do
static **Collector**<**CharSequence**,?,**String**>	**joining**()

static **Collector**<**CharSequence**,?,**String**>	**joining**(**CharSequence** delimi
static **Collector**<**CharSequence**,?,**String**>	**joining**(**CharSequence** delimi **CharSequence** prefix, **CharSe**
static <T,U,A,R> **Collector**<T,?,R>	**mapping**(**Function**<? super T, U> mapper, **Collector**<? supe U,A,R> downstream)
static <T> **Collector**<T,?,**Optional**<T>>	**maxBy**(**Comparator**<? super T>
static <T> **Collector**<T,?,**Optional**<T>>	**minBy**(**Comparator**<? super T>

static <T> **Collector**<T,?,**Map**<**Boolean**,**List**<T>>>	**partitioningBy**(**Predicate**<?
static <T,D,A> **Collector**<T,?,**Map**<**Boolean**,D>>	**partitioningBy**(**Predicate**<? super T> predicate, **Collect** T,A,D> downstream)
static <T> **Collector**<T,?,**Optional**<T>>	**reducing**(**BinaryOperator**<T>
static <T> **Collector**<T,?,T>	**reducing**(T identity, **Binary**

static <T,U> **Collector**<T,?,U>	**reducing**(U identity, **Functi** extends U> mapper, **BinaryOp**
static <T> **Collector**<T,?,**DoubleSummaryStatistics**>	**summarizingDouble**(**ToDoubleF** T> mapper)
static <T> **Collector**<T,?,**IntSummaryStatistics**>	**summarizingInt**(**ToIntFunctio** T> mapper)

static <T> **Collector**<T,?,**LongSummaryStatistics**>	**summarizingLong**(**ToLongFunct** T> mapper)
static <T> **Collector**<T,?,**Double**>	**summingDouble**(**ToDoubleFunct** T> mapper)
static <T> **Collector**<T,?,**Integer**>	**summingInt**(**ToIntFunction**<?
static <T> **Collector**<T,?,**Long**>	**summingLong**(**ToLongFunction**<
static <T,C extends **Collection**<T>> **Collector**<T,?,C>	**toCollection**(**Supplier**<C> co
static <T,K,U> **Collector**<T,?,**ConcurrentMap**<K,U>>	**toConcurrentMap**(**Function**<? K> keyMapper, **Function**<? su U> valueMapper)

static <T,K,U> **Collector**<T,?,**ConcurrentMap**<K,U>>	**toConcurrentMap**(**Function**<? K> keyMapper, **Function**<? su U> valueMapper, **BinaryOperator**<U> mergeFunc
static <T,K,U,M extends **ConcurrentMap**<K,U>> **Collector**<T,?,M>	**toConcurrentMap**(**Function**<? K> keyMapper, **Function**<? su U> valueMapper, **BinaryOperator**<U> mergeFunc **Supplier**<M> mapSupplier)
static <T> **Collector**<T,?,**List**<T>>	**toList**()
static <T,K,U> **Collector**<T,?,**Map**<K,U>>	

	toMap(**Function**<? super T,? K> keyMapper, **Function**<? su U> valueMapper)
static <T,K,U> **Collector**<T,?,**Map**<K,U>>	**toMap**(**Function**<? super T,? K> keyMapper, **Function**<? su U> valueMapper, **BinaryOperator**<U> mergeFunc
static <T,K,U,M extends **Map**<K,U>> **Collector**<T,?,M>	**toMap**(**Function**<? super T,? K> keyMapper, **Function**<? su U> valueMapper, **BinaryOperator**<U> mergeFunc **Supplier**<M> mapSupplier)
static <T> **Collector**<T,?,**Set**<T>>	**toSet**()

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail

toCollection

```
public static <T,C extends Collection<T>> Collector<T,?,C> toCollection(Supplier<
```

Returns a `Collector` that accumulates the input elements into a new `Collection`, in encounter order. The `Collection` is created by the provided factory.

Type Parameters:

`T - the type of the input elements`

`C - the type of the resulting Collection`

Parameters:

`collectionFactory - a Supplier which returns a new, empty Collection of the appropriate type`

Returns:

`a Collector which collects all the input elements into a Collection, in encounter order`

toList

```
public static <T> Collector<T,?,List<T>> toList()
```

Returns a `Collector` that accumulates the input elements into a new `List`. There are no guarantees on the type, mutability, serializability, or thread-safety of the `List` returned; if more control over the returned `List` is required, use `toCollection(Supplier)`.

Type Parameters:

`T - the type of the input elements`

Returns:

`a Collector which collects all the input elements into a List, in encounter order`

toSet

```
public static <T> Collector<T,?,Set<T>> toSet()
```

Returns a `Collector` that accumulates the input elements into a new `Set`. There are no guarantees on the type, mutability, serializability, or thread-safety of the `Set` returned; if more control over the returned `Set` is required, use `toCollection(Supplier)`.

This is an `unordered` Collector.

Type Parameters:

`T - the type of the input elements`

Returns:

`a Collector which collects all the input elements into a Set`

joining

```
public static Collector<CharSequence,?,String> joining()
```

Returns a `Collector` that concatenates the input elements into a `String`, in encounter order.

Returns:

`a Collector that concatenates the input elements into a String, in encounter order`

joining

```
public static Collector<CharSequence,?,String> joining(CharSequence delimiter)
```

Returns a `Collector` that concatenates the input elements, separated by the specified delimiter, in encounter order.

Parameters:

`delimiter - the delimiter to be used between each element`

Returns:

`A Collector which concatenates CharSequence elements, separated by the specified delimiter, in encounter order`

joining

```
public static Collector<CharSequence,?,String> joining(CharSequence delimiter,
                                                       CharSequence prefix,
                                                       CharSequence suffix)
```

Returns a `Collector` that concatenates the input elements, separated by the specified delimiter, with the specified prefix and suffix, in encounter order.

Parameters:

`delimiter - the delimiter to be used between each element`

`prefix - the sequence of characters to be used at the beginning of the joined result`

`suffix - the sequence of characters to be used at the end of the joined result`

Returns:

A Collector which concatenates CharSequence elements, separated by the specified delimiter, in encounter order

mapping

```
public static <T,U,A,R> Collector<T,?,R> mapping(Function<? super T,? extends U>
                                                 Collector<? super U,A,R> downstr
```

Adapts a `Collector` accepting elements of type `U` to one accepting elements of type `T` by applying a mapping function to each input element before accumulation.

API Note:

The mapping() collectors are most useful when used in a multi-level reduction, such as downstream of a groupingBy or partitioningBy. For example, given a stream of Person, to accumulate the set of last names in each city:

```
Map<City, Set<String>> lastNamesByCity
    = people.stream().collect(groupingBy(Person::getCity,
                                         mapping(Person::getLastName, toSet(
```

Type Parameters:

`T` - the type of the input elements

`U` - type of elements accepted by downstream collector

`A` - intermediate accumulation type of the downstream collector

`R` - result type of collector

Parameters:

`mapper` - a function to be applied to the input elements

`downstream` - a collector which will accept mapped values

Returns:

a collector which applies the mapping function to the input elements and provides the mapped results to the downstream collector

collectingAndThen

```
public static <T,A,R,RR> Collector<T,A,RR> collectingAndThen(Collector<T,A,R> dow
                                                             Function<R,RR> finis
```

Adapts a Collector to perform an additional finishing transformation. For example, one could adapt the toList() collector to always produce an immutable list with:

```
List<String> people
    = people.stream().collect(collectingAndThen(toList(), Collections::unmod
```

Type Parameters:

T - the type of the input elements

A - intermediate accumulation type of the downstream collector

R - result type of the downstream collector

RR - result type of the resulting collector

Parameters:

downstream - a collector

finisher - a function to be applied to the final result of the downstream collector

Returns:

a collector which performs the action of the downstream collector, followed by an additional finishing step

counting

```
public static <T> Collector<T,?,Long> counting()
```

Returns a Collector accepting elements of type T that counts the number of input elements. If no elements are present, the result is 0.

Implementation Requirements:

This produces a result equivalent to:

```
reducing(0L, e -> 1L, Long::sum)
```

Type Parameters:

T - the type of the input elements

Returns:

a Collector that counts the input elements

minBy

```
public static <T> Collector<T,?,Optional<T>> minBy(Comparator<? super T> comparat
```

Returns a `Collector` that produces the minimal element according to a given `Comparator`, described as an `Optional<T>`.

Implementation Requirements:

This produces a result equivalent to:

```
reducing(BinaryOperator.minBy(comparator))
```

Type Parameters:

`T` - the type of the input elements

Parameters:

`comparator` - a `Comparator` for comparing elements

Returns:

a `Collector` that produces the minimal value

maxBy

```
public static <T> Collector<T,?,Optional<T>> maxBy(Comparator<? super T> comparat
```

Returns a `Collector` that produces the maximal element according to a given `Comparator`, described as an `Optional<T>`.

Implementation Requirements:

This produces a result equivalent to:

```
reducing(BinaryOperator.maxBy(comparator))
```

Type Parameters:

`T` - the type of the input elements

Parameters:

`comparator` - a `Comparator` for comparing elements

Returns:

a `Collector` that produces the maximal value

summingInt

```
public static <T> Collector<T,?,Integer> summingInt(ToIntFunction<? super T> mapp
```

Returns a `Collector` that produces the sum of a integer-valued function applied to the input elements. If no elements are present, the result is 0.

Type Parameters:

`T - the type of the input elements`

Parameters:

`mapper - a function extracting the property to be summed`

Returns:

`a Collector that produces the sum of a derived property`

summingLong

```
public static <T> Collector<T,?,Long> summingLong(ToLongFunction<? super T> mappe
```

Returns a `Collector` that produces the sum of a long-valued function applied to the input elements. If no elements are present, the result is 0.

Type Parameters:

`T - the type of the input elements`

Parameters:

`mapper - a function extracting the property to be summed`

Returns:

`a Collector that produces the sum of a derived property`

summingDouble

```
public static <T> Collector<T,?,Double> summingDouble(ToDoubleFunction<? super T>
```

Returns a `Collector` that produces the sum of a double-valued function applied to the input elements. If no elements are present, the result is 0.

The sum returned can vary depending upon the order in which values are recorded, due to accumulated rounding error in addition of values of differing magnitudes. Values sorted by increasing absolute magnitude tend to yield more accurate results. If any recorded value is a `NaN` or the sum is at any point a `NaN` then the sum will be `NaN`.

Type Parameters:

`T - the type of the input elements`

Parameters:

mapper - a function extracting the property to be summed

Returns:

a Collector that produces the sum of a derived property

averagingInt

```
public static <T> Collector<T,?,Double> averagingInt(ToIntFunction<? super T> map
```

Returns a `Collector` that produces the arithmetic mean of an integer-valued function applied to the input elements. If no elements are present, the result is 0.

Type Parameters:

T - the type of the input elements

Parameters:

mapper - a function extracting the property to be summed

Returns:

a Collector that produces the sum of a derived property

averagingLong

```
public static <T> Collector<T,?,Double> averagingLong(ToLongFunction<? super T> m
```

Returns a `Collector` that produces the arithmetic mean of a long-valued function applied to the input elements. If no elements are present, the result is 0.

Type Parameters:

T - the type of the input elements

Parameters:

mapper - a function extracting the property to be summed

Returns:

a Collector that produces the sum of a derived property

averagingDouble

```
public static <T> Collector<T,?,Double> averagingDouble(ToDoubleFunction<? super
```

Returns a `Collector` that produces the arithmetic mean of a double-valued function applied to the input elements. If no elements are present, the result is 0.

The average returned can vary depending upon the order in which values are recorded, due to accumulated rounding error in addition of values of differing magnitudes. Values sorted by increasing absolute magnitude tend to yield more accurate results. If any recorded value is a NaN or the sum is at any point a NaN then the average will be NaN.

Implementation Note:

The double format can represent all consecutive integers in the range -2^{53} to 2^{53}. If the pipeline has more than 2^{53} values, the divisor in the average computation will saturate at 2^{53}, leading to additional numerical errors.

Type Parameters:

T - the type of the input elements

Parameters:

mapper - a function extracting the property to be summed

Returns:

a Collector that produces the sum of a derived property

reducing

```
public static <T> Collector<T,?,T> reducing(T identity,
                                            BinaryOperator<T> op)
```

Returns a Collector which performs a reduction of its input elements under a specified BinaryOperator using the provided identity.

API Note:

The reducing() collectors are most useful when used in a multi-level reduction, downstream of groupingBy or partitioningBy. To perform a simple reduction on a stream, use Stream.reduce(Object, BinaryOperator)} instead.

Type Parameters:

T - element type for the input and output of the reduction

Parameters:

identity - the identity value for the reduction (also, the value that is returned when there are no input elements)

op - a BinaryOperator<T> used to reduce the input elements

Returns:

a Collector which implements the reduction operation

See Also:

reducing(BinaryOperator), reducing(Object, Function, BinaryOperator)

reducing

```
public static <T> Collector<T,?,Optional<T>> reducing(BinaryOperator<T> op)
```

Returns a `Collector` which performs a reduction of its input elements under a specified `BinaryOperator`. The result is described as an `Optional<T>`.

API Note:

The reducing() collectors are most useful when used in a multi-level reduction, downstream of groupingBy or partitioningBy. To perform a simple reduction on a stream, use `Stream.reduce(BinaryOperator)` instead.

For example, given a stream of Person, to calculate tallest person in each city:

```
Comparator<Person> byHeight = Comparator.comparing(Person::getHeight);
Map<City, Person> tallestByCity
    = people.stream().collect(groupingBy(Person::getCity, reducing(BinaryOpe
```

Type Parameters:

`T` - element type for the input and output of the reduction

Parameters:

`op` - a `BinaryOperator<T>` used to reduce the input elements

Returns:

a `Collector` which implements the reduction operation

See Also:

`reducing(Object, BinaryOperator)`, `reducing(Object, Function, BinaryOperator)`

reducing

```
public static <T,U> Collector<T,?,U> reducing(U identity,
                                              Function<? super T,? extends U> map
                                              BinaryOperator<U> op)
```

Returns a `Collector` which performs a reduction of its input elements under a specified mapping function and `BinaryOperator`.
This is a generalization of `reducing(Object, BinaryOperator)` which allows a transformation of the elements before reduction.

API Note:

The reducing() collectors are most useful when used in a multi-level reduction, downstream of groupingBy or partitioningBy. To

perform a simple map-reduce on a stream, use Stream.map(Function) and Stream.reduce(Object, BinaryOperator) instead.

For example, given a stream of Person, to calculate the longest last name of residents in each city:

```
Comparator<String> byLength = Comparator.comparing(String::length);
Map<City, String> longestLastNameByCity
    = people.stream().collect(groupingBy(Person::getCity,
                                         reducing(Person::getLastName, Binar
```

Type Parameters:

T - the type of the input elements

U - the type of the mapped values

Parameters:

identity - the identity value for the reduction (also, the value that is returned when there are no input elements)

mapper - a mapping function to apply to each input value

op - a BinaryOperator<U> used to reduce the mapped values

Returns:

a Collector implementing the map-reduce operation

See Also:

reducing(Object, BinaryOperator), reducing(BinaryOperator)

groupingBy

public static <T,K> Collector<T,?,Map<K,List<T>>> groupingBy(Function<? super T,?

Returns a Collector implementing a "group by" operation on input elements of type T, grouping elements according to a classification function, and returning the results in a Map.

The classification function maps elements to some key type K. The collector produces a Map<K, List<T>> whose keys are the values resulting from applying the classification function to the input elements, and whose corresponding values are Lists containing the input elements which map to the associated key under the classification function.

There are no guarantees on the type, mutability, serializability, or thread-safety of the Map or List objects returned.

Implementation Requirements:

This produces a result similar to:

```
groupingBy(classifier, toList());
```

Implementation Note:

The returned Collector is not concurrent. For parallel stream pipelines, the combiner function operates by merging the keys from one map into another, which can be an expensive operation. If preservation of the order in which elements appear in the resulting Map collector is not required, using groupingByConcurrent(Function) may offer better parallel performance.

Type Parameters:

T - the type of the input elements

K - the type of the keys

Parameters:

classifier - the classifier function mapping input elements to keys

Returns:

a Collector implementing the group-by operation

See Also:

groupingBy(Function, Collector), groupingBy(Function, Supplier, Collector), groupingByConcurrent(Function)

groupingBy

```
public static <T,K,A,D> Collector<T,?,Map<K,D>> groupingBy(Function<? super T,? e
                                                           Collector<? super T,A,
```

Returns a Collector implementing a cascaded "group by" operation on input elements of type T, grouping elements according to a classification function, and then performing a reduction operation on the values associated with a given key using the specified downstream Collector.

The classification function maps elements to some key type K. The downstream collector operates on elements of type T and produces a result of type D. The resulting collector produces a Map<K, D>.

There are no guarantees on the type, mutability, serializability, or thread-safety of the Map returned.

For example, to compute the set of last names of people in each city:

```
Map<City, Set<String>> namesByCity
    = people.stream().collect(groupingBy(Person::getCity,
```

```
                                                    mapping(Person::getLastName, toSet(
```

Implementation Note:

The returned Collector is not concurrent. For parallel stream pipelines, the combiner function operates by merging the keys from one map into another, which can be an expensive operation. If preservation of the order in which elements are presented to the downstream collector is not required, using groupingByConcurrent(Function, Collector) may offer better parallel performance.

Type Parameters:

T - the type of the input elements

K - the type of the keys

A - the intermediate accumulation type of the downstream collector

D - the result type of the downstream reduction

Parameters:

classifier - a classifier function mapping input elements to keys

downstream - a Collector implementing the downstream reduction

Returns:

a Collector implementing the cascaded group-by operation

See Also:

groupingBy(Function), groupingBy(Function, Supplier, Collector), groupingByConcurrent(Function, Collector)

groupingBy

```
public static <T,K,D,A,M extends Map<K,D>> Collector<T,?,M> groupingBy(Function<?
                                                                       Supplier<M
                                                                       Collector<
```

Returns a Collector implementing a cascaded "group by" operation on input elements of type T, grouping elements according to a classification function, and then performing a reduction operation on the values associated with a given key using the specified downstream Collector. The Map produced by the Collector is created with the supplied factory function.

The classification function maps elements to some key type K. The downstream collector operates on elements of type T and produces a result of type D. The resulting collector produces a Map<K, D>.

For example, to compute the set of last names of people in each city, where the city names are sorted:

```
Map<City, Set<String>> namesByCity
    = people.stream().collect(groupingBy(Person::getCity, TreeMap::new,
                                         mapping(Person::getLastName, toSet(
```

Implementation Note:

The returned Collector is not concurrent. For parallel stream pipelines, the combiner function operates by merging the keys from one map into another, which can be an expensive operation. If preservation of the order in which elements are presented to the downstream collector is not required, using groupingByConcurrent(Function, Supplier, Collector) may offer better parallel performance.

Type Parameters:

T - the type of the input elements

K - the type of the keys

A - the intermediate accumulation type of the downstream collector

D - the result type of the downstream reduction

M - the type of the resulting Map

Parameters:

classifier - a classifier function mapping input elements to keys

downstream - a Collector implementing the downstream reduction

mapFactory - a function which, when called, produces a new empty Map of the desired type

Returns:

a Collector implementing the cascaded group-by operation

See Also:

groupingBy(Function, Collector), groupingBy(Function), groupingByConcurrent(Function, Supplier, Collector)

groupingByConcurrent

```
public static <T,K> Collector<T,?,ConcurrentMap<K,List<T>>> groupingByConcurrent(
```

Returns a concurrent Collector implementing a "group by" operation on input elements of type T, grouping elements according to a classification function.

This is a concurrent and unordered Collector.

The classification function maps elements to some key type K. The collector produces a ConcurrentMap<K, List<T>> whose keys are the values resulting

from applying the classification function to the input elements, and whose corresponding values are `Lists` containing the input elements which map to the associated key under the classification function.

There are no guarantees on the type, mutability, or serializability of the `Map` or `List` objects returned, or of the thread-safety of the `List` objects returned.

Implementation Requirements:

`This produces a result similar to:`

```
groupingByConcurrent(classifier, toList());
```

Type Parameters:

`T - the type of the input elements`

`K - the type of the keys`

Parameters:

`classifier - a classifier function mapping input elements to keys`

Returns:

`a concurrent, unordered Collector implementing the group-by operation`

See Also:

`groupingBy(Function)`, `groupingByConcurrent(Function, Collector)`, `groupingByConcurrent(Function, Supplier, Collector)`

groupingByConcurrent

```
public static <T,K,A,D> Collector<T,?,ConcurrentMap<K,D>> groupingByConcurrent(Fu
                                                                               Co
```

Returns a concurrent `Collector` implementing a cascaded "group by" operation on input elements of type `T`, grouping elements according to a classification function, and then performing a reduction operation on the values associated with a given key using the specified downstream `Collector`.

This is a `concurrent` and `unordered` Collector.

The classification function maps elements to some key type `K`. The downstream collector operates on elements of type `T` and produces a result of type `D`. The resulting collector produces a `Map<K, D>`.

For example, to compute the set of last names of people in each city, where the city names are sorted:

```
     ConcurrentMap<City, Set<String>> namesByCity
         = people.stream().collect(groupingByConcurrent(Person::getCity,
                                                        mapping(Person::getLastNa
```

Type Parameters:

`T` - the type of the input elements

`K` - the type of the keys

`A` - the intermediate accumulation type of the downstream collector

`D` - the result type of the downstream reduction

Parameters:

`classifier` - a classifier function mapping input elements to keys

`downstream` - a Collector implementing the downstream reduction

Returns:

a concurrent, unordered Collector implementing the cascaded group-by operation

See Also:

`groupingBy(Function, Collector)`, `groupingByConcurrent(Function)`, `groupingByConcurrent(Function, Supplier, Collector)`

groupingByConcurrent

```
public static <T,K,A,D,M extends ConcurrentMap<K,D>> Collector<T,?,M> groupingByC
```

Returns a concurrent `Collector` implementing a cascaded "group by" operation on input elements of type `T`, grouping elements according to a classification function, and then performing a reduction operation on the values associated with a given key using the specified downstream `Collector`. The `ConcurrentMap` produced by the Collector is created with the supplied factory function.

This is a `concurrent` and `unordered` Collector.

The classification function maps elements to some key type `K`. The downstream collector operates on elements of type `T` and produces a result of type `D`. The resulting collector produces a `Map<K, D>`.

For example, to compute the set of last names of people in each city, where the city names are sorted:

```
     ConcurrentMap<City, Set<String>> namesByCity
```

```
        = people.stream().collect(groupingBy(Person::getCity, ConcurrentSkipList
                                             mapping(Person::getLastName, toSet(
```

Type Parameters:

T - the type of the input elements

K - the type of the keys

A - the intermediate accumulation type of the downstream collector

D - the result type of the downstream reduction

M - the type of the resulting ConcurrentMap

Parameters:

classifier - a classifier function mapping input elements to keys

downstream - a Collector implementing the downstream reduction

mapFactory - a function which, when called, produces a new empty ConcurrentMap of the desired type

Returns:

a concurrent, unordered Collector implementing the cascaded group-by operation

See Also:

groupingByConcurrent(Function), groupingByConcurrent(Function, Collector), groupingBy(Function, Supplier, Collector)

partitioningBy

```
public static <T> Collector<T,?,Map<Boolean,List<T>>> partitioningBy(Predicate<?
```

Returns a `Collector` which partitions the input elements according to a `Predicate`, and organizes them into a `Map<Boolean, List<T>>`. There are no guarantees on the type, mutability, serializability, or thread-safety of the `Map` returned.

Type Parameters:

T - the type of the input elements

Parameters:

predicate - a predicate used for classifying input elements

Returns:

a Collector implementing the partitioning operation

See Also:

partitioningBy(Predicate, Collector)

partitioningBy

```
public static <T,D,A> Collector<T,?,Map<Boolean,D>> partitioningBy(Predicate<? su
                                                                   Collector<? su
```

Returns a `Collector` which partitions the input elements according to a `Predicate`, reduces the values in each partition according to another `Collector`, and organizes them into a `Map<Boolean, D>` whose values are the result of the downstream reduction.

There are no guarantees on the type, mutability, serializability, or thread-safety of the `Map` returned.

Type Parameters:

`T - the type of the input elements`

`A - the intermediate accumulation type of the downstream collector`

`D - the result type of the downstream reduction`

Parameters:

`predicate - a predicate used for classifying input elements`

`downstream - a Collector implementing the downstream reduction`

Returns:

`a Collector implementing the cascaded partitioning operation`

See Also:

`partitioningBy(Predicate)`

toMap

```
public static <T,K,U> Collector<T,?,Map<K,U>> toMap(Function<? super T,? extends
                                                    Function<? super T,? extends
```

Returns a `Collector` that accumulates elements into a `Map` whose keys and values are the result of applying the provided mapping functions to the input elements.

If the mapped keys contains duplicates (according to `Object.equals(Object)`), an `IllegalStateException` is thrown when the collection operation is performed. If the mapped keys may have duplicates, use `toMap(Function, Function, BinaryOperator)` instead.

API Note:

`It is common for either the key or the value to be the input elements. In this case, the utility method Function.identity() may be helpful. For example, the following produces a Map mapping students to their grade point average:`

```
Map<Student, Double> studentToGPA
    students.stream().collect(toMap(Functions.identity(),
                                    student -> computeGPA(student)));
```

And the following produces a Map mapping a unique identifier to students:

```
Map<String, Student> studentIdToStudent
    students.stream().collect(toMap(Student::getId,
                                    Functions.identity());
```

Implementation Note:

The returned Collector is not concurrent. For parallel stream pipelines, the combiner function operates by merging the keys from one map into another, which can be an expensive operation. If it is not required that results are inserted into the Map in encounter order, using toConcurrentMap(Function, Function) may offer better parallel performance.

Type Parameters:

T - the type of the input elements

K - the output type of the key mapping function

U - the output type of the value mapping function

Parameters:

keyMapper - a mapping function to produce keys

valueMapper - a mapping function to produce values

Returns:

a Collector which collects elements into a Map whose keys and values are the result of applying mapping functions to the input elements

See Also:

toMap(Function, Function, BinaryOperator), toMap(Function, Function, BinaryOperator, Supplier), toConcurrentMap(Function, Function)

toMap

```
public static <T,K,U> Collector<T,?,Map<K,U>> toMap(Function<? super T,? extends
                                                    Function<? super T,? extends
                                                    BinaryOperator<U> mergeFuncti
```

Returns a `Collector` that accumulates elements into a `Map` whose keys and values are the result of applying the provided mapping functions to the input elements.

If the mapped keys contains duplicates (according to `Object.equals(Object)`), the value mapping function is applied to each equal element, and the results are merged using the provided merging function.

API Note:

There are multiple ways to deal with collisions between multiple elements mapping to the same key. The other forms of toMap simply use a merge function that throws unconditionally, but you can easily write more flexible merge policies. For example, if you have a stream of Person, and you want to produce a "phone book" mapping name to address, but it is possible that two persons have the same name, you can do as follows to gracefully deals with these collisions, and produce a Map mapping names to a concatenated list of addresses:

```
Map<String, String> phoneBook
    people.stream().collect(toMap(Person::getName,
                                  Person::getAddress,
                                  (s, a) -> s + ", " + a));
```

Implementation Note:

The returned Collector is not concurrent. For parallel stream pipelines, the combiner function operates by merging the keys from one map into another, which can be an expensive operation. If it is not required that results are merged into the Map in encounter order, using `toConcurrentMap(Function, Function, BinaryOperator)` may offer better parallel performance.

Type Parameters:

T - the type of the input elements

K - the output type of the key mapping function

U - the output type of the value mapping function

Parameters:

keyMapper - a mapping function to produce keys

valueMapper - a mapping function to produce values

mergeFunction - a merge function, used to resolve collisions between values associated with the same key, as supplied to `Map.merge(Object, Object, BiFunction)`

Returns:

a Collector which collects elements into a Map whose keys are the result of applying a key mapping function to the input elements, and whose values are the result of applying a value mapping function to all input elements equal to the key and combining them using the merge function

See Also:

toMap(Function, Function), toMap(Function, Function, BinaryOperator, Supplier), toConcurrentMap(Function, Function, BinaryOperator)

toMap

```
public static <T,K,U,M extends Map<K,U>> Collector<T,?,M> toMap(Function<? super
                                                                Function<? super
                                                                BinaryOperator<U>
                                                                Supplier<M> mapSu
```

Returns a Collector that accumulates elements into a Map whose keys and values are the result of applying the provided mapping functions to the input elements.

If the mapped keys contains duplicates (according to Object.equals(Object)), the value mapping function is applied to each equal element, and the results are merged using the provided merging function. The Map is created by a provided supplier function.

Implementation Note:

The returned Collector is not concurrent. For parallel stream pipelines, the combiner function operates by merging the keys from one map into another, which can be an expensive operation. If it is not required that results are merged into the Map in encounter order, using toConcurrentMap(Function, Function, BinaryOperator, Supplier) may offer better parallel performance.

Type Parameters:

T - the type of the input elements

K - the output type of the key mapping function

U - the output type of the value mapping function

M - the type of the resulting Map

Parameters:

keyMapper - a mapping function to produce keys

valueMapper - a mapping function to produce values

mergeFunction - a merge function, used to resolve collisions between values associated with the same key, as supplied to Map.merge(Object, Object, BiFunction)

mapSupplier - a function which returns a new, empty Map into which the results will be inserted

Returns:

a Collector which collects elements into a Map whose keys are the result of applying a key mapping function to the input elements, and whose values are the result of applying a value mapping function to all input elements equal to the key and combining them using the merge function

See Also:

toMap(Function, Function), toMap(Function, Function, BinaryOperator), toConcurrentMap(Function, Function, BinaryOperator, Supplier)

toConcurrentMap

```
public static <T,K,U> Collector<T,?,ConcurrentMap<K,U>> toConcurrentMap(Function<
                                                                        Function<
```

Returns a concurrent Collector that accumulates elements into a ConcurrentMap whose keys and values are the result of applying the provided mapping functions to the input elements.

If the mapped keys contains duplicates (according to Object.equals(Object)), an IllegalStateException is thrown when the collection operation is performed. If the mapped keys may have duplicates, use toConcurrentMap(Function, Function, BinaryOperator) instead.

API Note:

It is common for either the key or the value to be the input elements. In this case, the utility method Function.identity() may be helpful. For example, the following produces a Map mapping students to their grade point average:

```
Map<Student, Double> studentToGPA
    students.stream().collect(toMap(Functions.identity(),
                                    student -> computeGPA(student)));
```

And the following produces a Map mapping a unique identifier to students:

```
Map<String, Student> studentIdToStudent
    students.stream().collect(toConcurrentMap(Student::getId,
                                              Functions.identity());
```

This is a concurrent and unordered Collector.

Type Parameters:

T - the type of the input elements

K - the output type of the key mapping function

U - the output type of the value mapping function

Parameters:

keyMapper - the mapping function to produce keys

valueMapper - the mapping function to produce values

Returns:

a concurrent, unordered Collector which collects elements into a ConcurrentMap whose keys are the result of applying a key mapping function to the input elements, and whose values are the result of applying a value mapping function to the input elements

See Also:

toMap(Function, Function), toConcurrentMap(Function, Function, BinaryOperator), toConcurrentMap(Function, Function, BinaryOperator, Supplier)

toConcurrentMap

```
public static <T,K,U> Collector<T,?,ConcurrentMap<K,U>> toConcurrentMap(Function<
                                                                        Function<
                                                                        BinaryOpe
```

Returns a concurrent Collector that accumulates elements into a ConcurrentMap whose keys and values are the result of applying the provided mapping functions to the input elements.

If the mapped keys contains duplicates (according to Object.equals(Object)), the value mapping function is applied to each equal element, and the results are merged using the provided merging function.

API Note:

There are multiple ways to deal with collisions between multiple elements mapping to the same key. The other forms of toConcurrentMap simply use a merge function that throws unconditionally, but you can easily write more flexible merge policies. For example, if you have a stream of Person, and you want to produce a "phone book" mapping name to address, but it is possible that two persons have the same name, you can do as follows to gracefully deals with these collisions, and produce a Map mapping names to a concatenated list of addresses:

```
Map<String, String> phoneBook
    people.stream().collect(toConcurrentMap(Person::getName,
                                            Person::getAddress,
                                            (s, a) -> s + ", " + a));
```

This is a concurrent and unordered Collector.

Type Parameters:

T - the type of the input elements

K - the output type of the key mapping function

U - the output type of the value mapping function

Parameters:

keyMapper - a mapping function to produce keys

valueMapper - a mapping function to produce values

mergeFunction - a merge function, used to resolve collisions between values associated with the same key, as supplied to Map.merge(Object, Object, BiFunction)

Returns:

a concurrent, unordered Collector which collects elements into a ConcurrentMap whose keys are the result of applying a key mapping function to the input elements, and whose values are the result of applying a value mapping function to all input elements equal to the key and combining them using the merge function

See Also:

toConcurrentMap(Function, Function), toConcurrentMap(Function, Function, BinaryOperator, Supplier), toMap(Function, Function, BinaryOperator)

toConcurrentMap

```
public static <T,K,U,M extends ConcurrentMap<K,U>> Collector<T,?,M> toConcurrentM
```

Returns a concurrent Collector that accumulates elements into a ConcurrentMap whose keys and values are the result of applying the provided mapping functions to the input elements.

If the mapped keys contains duplicates (according to Object.equals(Object)), the value mapping function is applied to each equal element, and the results are merged using the provided merging function. The ConcurrentMap is created by a provided supplier function.

This is a concurrent and unordered Collector.

Type Parameters:

T - the type of the input elements

K - the output type of the key mapping function

U - the output type of the value mapping function

M - the type of the resulting ConcurrentMap

Parameters:

keyMapper - a mapping function to produce keys

valueMapper - a mapping function to produce values

mergeFunction - a merge function, used to resolve collisions between values associated with the same key, as supplied to Map.merge(Object, Object, BiFunction)

mapSupplier - a function which returns a new, empty Map into which the results will be inserted

Returns:

a concurrent, unordered Collector which collects elements into a ConcurrentMap whose keys are the result of applying a key mapping function to the input elements, and whose values are the result of applying a value mapping function to all input elements equal to the key and combining them using the merge function

See Also:

toConcurrentMap(Function, Function), toConcurrentMap(Function, Function, BinaryOperator), toMap(Function, Function, BinaryOperator, Supplier)

summarizingInt

```
public static <T> Collector<T,?,IntSummaryStatistics> summarizingInt(ToIntFunctio
```

Returns a `Collector` which applies an `int`-producing mapping function to each input element, and returns summary statistics for the resulting values.

Type Parameters:

`T - the type of the input elements`

Parameters:

`mapper - a mapping function to apply to each element`

Returns:

`a Collector implementing the summary-statistics reduction`

See Also:

`summarizingDouble(ToDoubleFunction)`, `summarizingLong(ToLongFunction)`

summarizingLong

```
public static <T> Collector<T,?,LongSummaryStatistics> summarizingLong(ToLongFunc
```

Returns a `Collector` which applies an `long`-producing mapping function to each input element, and returns summary statistics for the resulting values.

Type Parameters:

`T - the type of the input elements`

Parameters:

`mapper - the mapping function to apply to each element`

Returns:

`a Collector implementing the summary-statistics reduction`

See Also:

`summarizingDouble(ToDoubleFunction)`, `summarizingInt(ToIntFunction)`

summarizingDouble

```
public static <T> Collector<T,?,DoubleSummaryStatistics> summarizingDouble(ToDoub
```

Returns a `Collector` which applies an `double`-producing mapping function to each input element, and returns summary statistics for the resulting values.

Type Parameters:

`T - the type of the input elements`

Parameters:

mapper - a mapping function to apply to each element

Returns:

a Collector implementing the summary-statistics reduction

See Also:

summarizingLong(ToLongFunction), summarizingInt(ToIntFunction)

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.